

DAY 11

Task 1:

What is an Actuator in Spring boot?

- Spring Boot Actuator is a module that provides production-ready features to monitor and manage your application.
It exposes endpoints like /health, /metrics, and /info to check application status and performance.

Task 2:

Advantages of Actuator

- Monitors application health and shows if the application is running properly.
- Provides performance metrics such as memory usage, CPU usage, and request statistics.
- Helps in faster troubleshooting by exposing useful runtime information.
- Reduces development effort with built-in monitoring and management endpoints.
- Improves application maintenance by giving real-time insights into system behavior.

Task 3:

What are the commonly used Actuator end points?

- **/actuator/health** – Shows the health status of the application.
- **/actuator/info** – Displays application information such as version and custom details.
- **/actuator/metrics** – Provides application performance metrics.
- **/actuator/beans** – Lists all Spring beans loaded in the application.
- **/actuator/mappings** – Shows all available request mappings (URLs).
- **/actuator/shutdown** – Gracefully shuts down the application (if enabled).

Task 4:

Create a Small demo of Actuator to know the functionality

- Create a project spring-boot-actuator-demo

SpringbootActuatorDemoApplication.java

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringbootActuatorDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringbootActuatorDemoApplication.class, args);
    }

}
```

DemoController.java

```
package com.example;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class DemoController {

    @GetMapping("/welcome")
    public String welcome() {
        return "Welcome to Actuator Demo";
    }
}
```

dependencies

```
<dependencies>

    <!-- Web Dependency -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

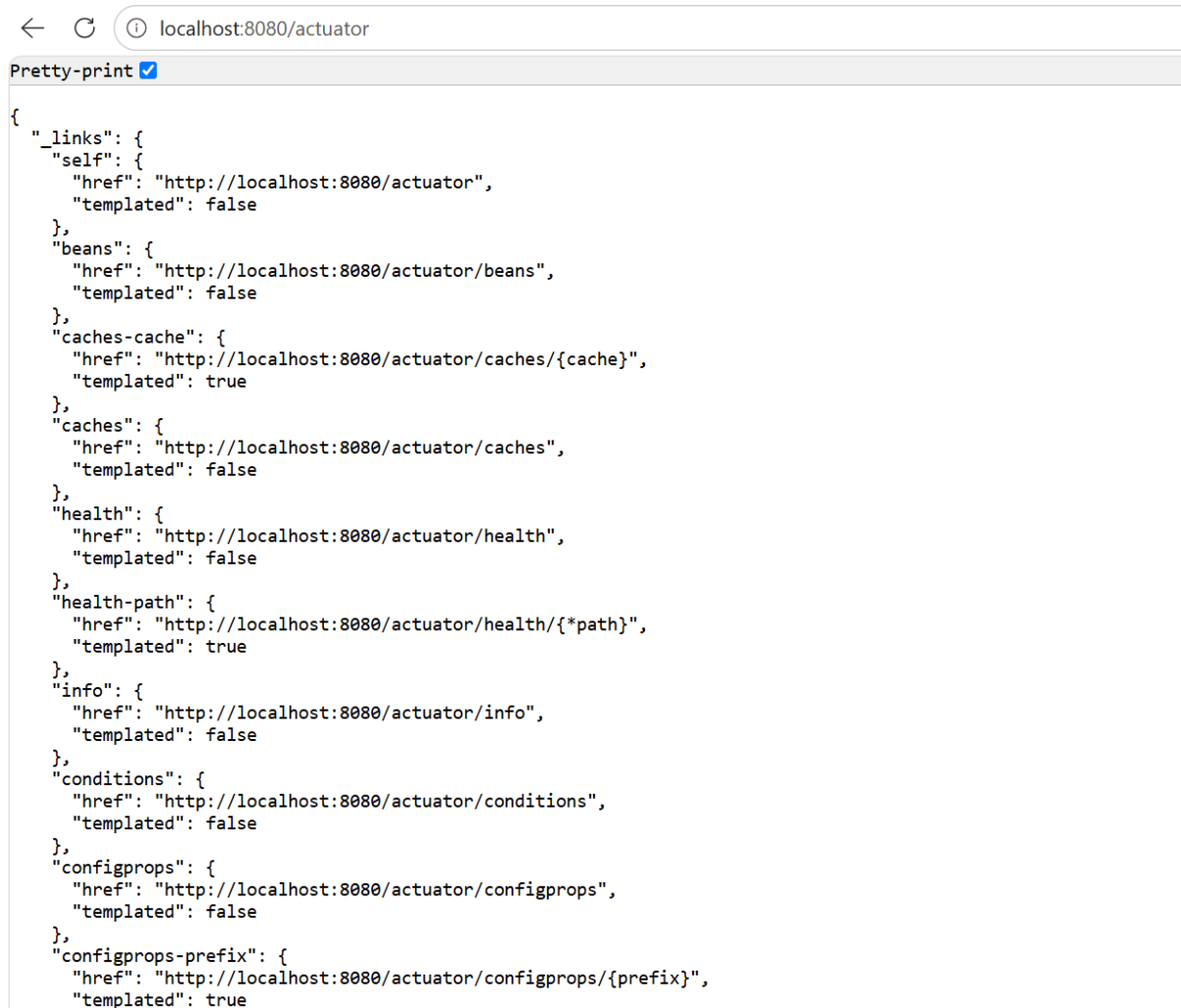
    <!-- Actuator Dependency -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>

</dependencies>
```

Application.properties

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

Output



```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8080/actuator/beans",
      "templated": false
    },
    "caches-cache": {
      "href": "http://localhost:8080/actuator/caches/{cache}",
      "templated": true
    },
    "caches": {
      "href": "http://localhost:8080/actuator/caches",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8080/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8080/actuator/health/{*path}",
      "templated": true
    },
    "info": {
      "href": "http://localhost:8080/actuator/info",
      "templated": false
    },
    "conditions": {
      "href": "http://localhost:8080/actuator/conditions",
      "templated": false
    },
    "configprops": {
      "href": "http://localhost:8080/actuator/configprops",
      "templated": false
    },
    "configprops-prefix": {
      "href": "http://localhost:8080/actuator/configprops/{prefix}",
      "templated": true
    }
  }
}
```

Task 5:

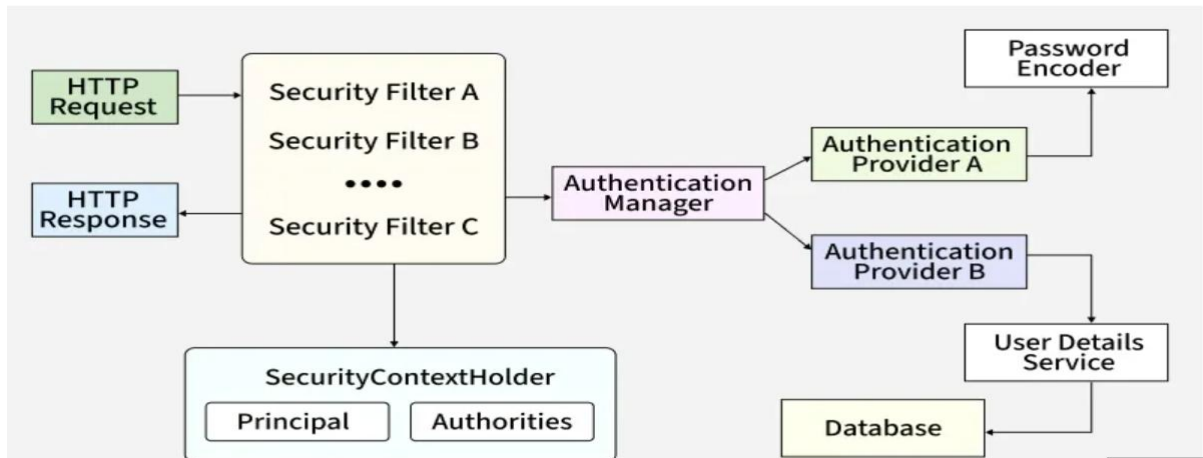
What is Spring Security Architecture?

Spring Security Architecture defines the components and workflow used to authenticate users and authorize access to application resources. It secures web applications by processing requests through a chain of security filters before they reach the application.

- Secures web applications using a filter chain.
- Handles authentication and authorization.
- Supports multiple login methods (Form, JWT, OAuth2, LDAP).

Task 6:

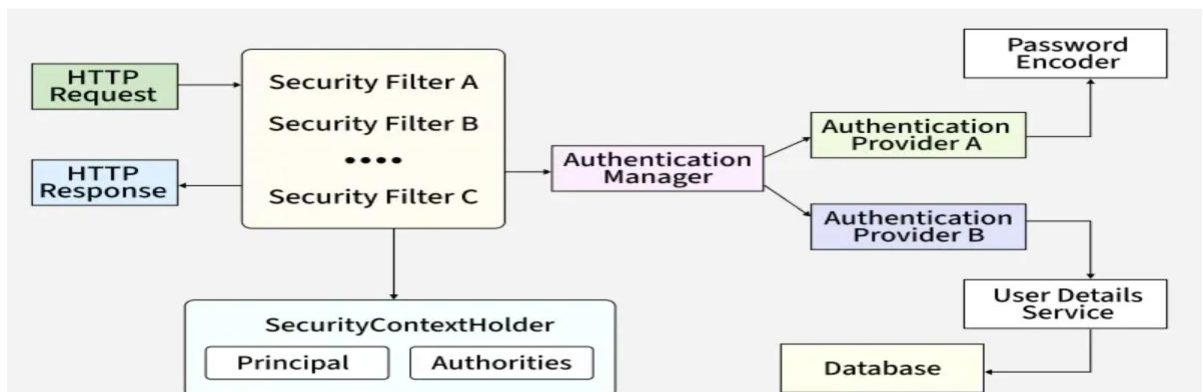
What are the core components of Spring security architecture



1. **Security Filter Chain** - Intercepts every incoming HTTP request and applies security checks such as authentication and authorization.
2. **Authentication Manager** - Receives authentication requests and delegates them to the appropriate Authentication Provider.
3. **Authentication Provider** - Validates user credentials and determines whether authentication is successful.
4. **UserDetailsService** - Loads user information (username, password, roles) from a database or other data source.
5. **PasswordEncoder** - Encrypts and verifies passwords securely during authentication.
6. **SecurityContextHolder** - Stores the authenticated user's details, including Principal and Authorities, for the current session/request.
7. **Authorization Mechanism** - Checks user roles and permissions to determine access to protected resources.

Task 7:

Can you create a diagrammatic representation of Spring Security architecture



Task 8:

What is a Security filter chain? Give example code

- A Security Filter Chain is a collection of security filters that intercept every HTTP request before it reaches the controller. It handles tasks such as authentication, authorization, session management, and exception handling.

```
- @Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain
    securityFilterChain(HttpSecurity http) throws Exception
    {

        http.csrf(
            csrf
            -> csrf.disable()) // Disable CSRF for APIs
        .authorizeHttpRequests(
            auth
            -> auth.requestMatchers("/public/**")
            .permitAll()
            .anyRequest()
            .authenticated())
        .httpBasic(); // Enable Basic Authentication

        return http.build();
    }
}
```

Task 9:

What is an Authentication Manager? Give example code

- Core component responsible for handling user authentication
- Delegates authentication requests to one or more AuthenticationProvider instances
- Follows the Strategy design pattern, allowing multiple authentication mechanisms such as DB, LDAP, JWT, and OAuth2

```
• @Configuration
public class AuthManagerConfig {

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration configuration)
        throws Exception
    {

        return configuration.getAuthenticationManager();
    }
}
```

Task 10:

What are Authentication Providers? Give example and code

- Authentication Providers are the components responsible for validating user credentials
- They process authentication requests coming from the AuthenticationManager
- Different providers support different authentication mechanisms

```
@Configuration
public class ProviderConfig {

    @Bean
    public DaoAuthenticationProvider authenticationProvider(
        UserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder)
    {

        DaoAuthenticationProvider provider
            = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder);

        return provider;
    }
}
```

Task 11:

What is UserDetailsService? Give example

- Loads user-specific data (username, password, roles) from a data source like a database.
- Returns a UserDetails object.
- Used primarily by providers like DaoAuthenticationProvider.

Example

```
@Configuration
public class UserConfig {

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder encoder) {
        return new InMemoryUserDetailsManager(
            User.withUsername("john")
                .password(encoder.encode("password"))
                .roles("USER")
                .build(),
            User.withUsername("admin")
                .password(encoder.encode("admin123"))
                .roles("ADMIN")
        );
    }
}
```

```
        .build()
    );
}
```

Task 12:

What is Password encoder? Give example

- Ensures secure password storage and validation.
- Encodes raw passwords into secure hashes before saving/validation.

Example:

```
@Configuration
public class PasswordConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // Strong hashing
    }
}
```

Task 13:

What is SecurityContextHolder?

SecurityContextHolder is a Spring Security class that stores the details of the currently authenticated user during a request.

It contains:

- **Principal** → Logged-in user information
- **Authorities** → User roles/permissions
- **Authentication** → Authentication details

Task 14:

Finally give the security Architecture internal functionality? How actually it works internally... step by step if you can explain

- A client sends an HTTP request to the application.
- The request passes through the Security Filter Chain where multiple security filters are applied.
- The Authentication Manager receives the request and delegates authentication to the appropriate Authentication Provider.
- The Authentication Provider validates credentials using UserDetailsService and PasswordEncoder (if required).

- On successful authentication, user details are stored in SecurityContextHolder.
- Authorization is performed using the stored principal and authorities to check access permissions.
- If all checks pass, the request reaches the controller and an HTTP response is returned.

Task 15:

What is Thymeleaf? Explain? Why is it used?

Thymeleaf is a modern Java template engine used to create dynamic web pages in Spring Boot applications. It allows developers to combine HTML with data from Java code and display it in the browser.

Thymeleaf is used for:

1. To create dynamic web pages with data from the backend.
2. To easily integrate with Spring Boot and Spring MVC.
3. To display user data, forms, tables, and reports in HTML pages.
4. To build user-friendly web applications using reusable templates.

Task 16:

Create a demo on thymeleaf .

- Create a project SpringBoot_Thymeleaf_Demo

ThymeleafDemoApplication.java

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ThymeleafDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(ThymeleafDemoApplication.class, args);
    }

}
```

HomeController.java

```
package com.example;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {
```

```

@GetMapping("/home")
public String home(Model model) {

    model.addAttribute("name", "Monika");
    model.addAttribute("course", "Spring Boot");

    return "home";
}
}

```

dependencies

```

<dependencies>

<!-- Spring Web -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Thymeleaf -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

</dependencies>

```

- Create a template folder under resource
- Inside the template create a home.html

```

- <!DOCTYPE html>
  <html xmlns:th="http://www.thymeleaf.org">

    <head>
        <title>Thymeleaf Demo</title>
    </head>

    <body>

        <h1>Welcome to Thymeleaf</h1>

        <p>Name: <span th:text="{name}"></span></p>

        <p>Course: <span th:text="{course}"></span></p>

    </body>

  </html>

```

Application.properties

```
spring.thymeleaf.cache=false
```

OUTPUT

Welcome to Thymeleaf

Name: Monika

Course: Spring Boot

Task 17:

What do you think are the advantages of using thymeleaf?

- Easy integration with Spring Boot
- Creates dynamic web pages
- Uses standard HTML templates
- Simple and easy to learn
- Supports form handling
- Improves code readability
- Enables reusable page layouts
- Reduces development effort
- Supports data binding with backend objects
- Provides better maintainability of web applications

Task 18:

What is Caching in Springboot?

- Caching is a technique used to store frequently accessed data in memory so that future requests can be served faster without repeatedly fetching data from the database.

Task 19:

What are the features of Caching in SpringBoot?

- ☐ Improves application performance by reducing data retrieval time.
- ☐ Reduces database calls by storing frequently accessed data.
- ☐ Faster response times for repeated requests.
- ☐ Supports multiple cache providers such as Redis, Caffeine, and Hazelcast.
- ☐ Annotation-based configuration using `@Cacheable`, `@CachePut`, and `@CacheEvict`.
- ☐ Automatic cache management through Spring Cache Abstraction.
- ☐ Supports cache **eviction** to remove outdated data.

Task 20:

What are the types of Caching? explain each type.

1. **CDN Caching** - is a group of distributed servers that speed up the delivery of web content by bringing it closer to where users are. Data centers across the globe use caching, to deliver internet content to a web-enabled device or browser more quickly through a server near you, reducing the load on an application origin and improving the user experience.
2. **Database Caching** - Database caching improves scalability by distributing query workload from the backend to multiple front-end systems. It allows flexibility in the processing of data. It can significantly reduce latency and increase throughput for read-heavy application workloads by avoiding, querying a database too much.
3. **In-Memory Caching** - In-Memory Caching increases the performance of the application. An in-memory cache is a common query store, therefore, relieves databases of reading workloads. Redis is distributed, and advanced caching tool that allows backup and restores facilities. In-memory Cache provides query functionality on top of caching.
4. **Web server Caching** - Web server caching stores data, such as a copy of a web page served by a web server. It is cached or stored the first time a user visits the page and when the next time a user requests the same page, the content will be delivered from the cache, which helps keep the origin server from getting overloaded. It enhances page delivery speed significantly and reduces the work needed to be done by the backend server.

Task 21:

List down annotations of Caching with usage.

@EnableCaching

- **Usage:** Enables caching support in a Spring Boot application.

```
@SpringBootApplication
@EnableCaching
public class DemoApplication {
}
```

@Cacheable

- **Usage:** Stores the method result in the cache and returns the cached value for subsequent calls.

```
@Cacheable("employees")
public Employee getEmployeeById(Long id) {
    return employeeRepository.findById(id).get();
}
```

@CachePut

- **Usage:** Updates the cache whenever the method is executed.

```
@CachePut(value = "employees", key = "#employee.id")
public Employee updateEmployee(Employee employee) {
```

```
return employeeRepository.save(employee);
}
```

@CacheEvict

- **Usage:** Removes data from the cache.

```
@CacheEvict(value = "employees", key = "#id")
public void deleteEmployee(Long id) {
    employeeRepository.deleteById(id);
}
```

@Caching

- **Usage:** Combines multiple caching operations in a single annotation

```
@Caching(
    put = {
        @CachePut(value = "employees", key = "#employee.id")
    },
    evict = {
        @CacheEvict(value = "employeeList", allEntries = true)
    }
)
public Employee updateEmployee(Employee employee) {
    return employeeRepository.save(employee);
}
```

BANKING MOCKITO

- Create a project called BankingMockitoProject

BankAccount

```
package com.example;
public class BankAccount {

    private String accountNumber;
    private double balance;

    public BankAccount(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
```

```
        this.balance = balance;
    }
}
```

BankRepository.java

```
package com.example;
public interface BankRepository {

    BankAccount findByAccountNumber(String accountNumber);

    void save(BankAccount account);
}
```

BankRepository.java

```
package com.example;
public class BankService {

    private BankRepository repository;

    public BankService(BankRepository repository) {
        this.repository = repository;
    }

    public void deposit(String accountNumber, double amount) {

        BankAccount account =
            repository.findByAccountNumber(accountNumber);

        account.setBalance(
            account.getBalance() + amount);

        repository.save(account);
    }

    public void withdraw(String accountNumber,
                        double amount) {

        BankAccount account =
            repository.findByAccountNumber(accountNumber);

        if (account.getBalance() < amount) {
            throw new IllegalArgumentException(
                "Insufficient funds");
        }

        account.setBalance(
            account.getBalance() - amount);

        repository.save(account);
    }
}
```

BankServiceTest.java

```
import com.example.BankAccount;
import com.example.BankRepository;
import com.example.BankService;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;

import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class BankServiceTest {

    @Test
    void testDeposit() {

        BankRepository mockRepo =
            mock(BankRepository.class);

        BankAccount account =
            new BankAccount("123", 500.0);

        when(mockRepo.findByAccountNumber("123"))
            .thenReturn(account);

        BankService service =
            new BankService(mockRepo);

        service.deposit("123", 200.0);

        assertEquals(
            700.0,
            account.getBalance());

        verify(mockRepo).save(account);
    }

    @Test
    void testWithdrawSuccess() {

        BankRepository mockRepo =
            mock(BankRepository.class);

        BankAccount account =
            new BankAccount("123", 500.0);

        when(mockRepo.findByAccountNumber("123"))
            .thenReturn(account);

        BankService service =
            new BankService(mockRepo);

        service.withdraw("123", 200.0);

        assertEquals(
            300.0,
            account.getBalance());
    }
}
```

```

        verify(mockRepo).save(account);
    }

    @Test
    void testWithdrawInsufficientFunds() {

        BankRepository mockRepo =
            mock(BankRepository.class);

        BankAccount account =
            new BankAccount("123", 100.0);

        when(mockRepo.findByAccountNumber("123"))
            .thenReturn(account);

        BankService service =
            new BankService(mockRepo);

        Exception ex =
            assertThrows(
                IllegalArgumentException.class,
                () -> service.withdraw("123", 200.0));

        assertEquals(
            "Insufficient funds",
            ex.getMessage());

        verify(mockRepo, never()).save(account);
    }
}

```

dependency

```

<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
</dependency>

<!-- Mockito -->
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>5.12.0</version>
    <scope>test</scope>
</dependency>

<!-- Mockito Extension for JUnit5 -->
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.12.0</version>
    <scope>test</scope>
</dependency>

```

Task 22:

What do you think is `@InjectMocks` annotation does?

- `@InjectMocks` tells Mockito to create an instance of the class being tested and automatically inject the required mock dependencies into it.
- `@Mock` creates fake dependencies, and `@InjectMocks` puts those fake dependencies into the class under test.

Task 23:

Write about `@spy` annotation and explain the functionality in a small piece of code

`@Spy` is used when you want to create a **partial mock** of a real object. Unlike `@Mock`, which creates a completely fake object, `@Spy` keeps the actual behavior of the object but allows you to stub or verify specific methods.

```
import org.junit.jupiter.api.Test;
import org.mockito.Spy;

import java.util.ArrayList;
import java.util.List;

import static org.mockito.Mockito.*;

class SpyExampleTest {

    @Spy
    List<String> list = new ArrayList<>();

    @Test
    void testSpy() {
        list.add("Apple");
        list.add("Banana");

        verify(list).add("Apple");

        System.out.println(list.size()); // Output: 2
    }
}
```

Task 24:

Why is `@Mock` annotation used?

- `@Mock` is used to create a **fake object** of a class or interface during testing. It helps isolate the class under test by replacing real dependencies with controlled, simulated behavior.

Task 25:

What is Partial mocking? Which annotation is used for partial mocking?

Partial Mocking means using a real object while mocking only specific methods of that object. The actual methods run by default, but you can override selected methods when needed.

- **@Spy**

Task 26:

For the code given what is **@verify** annotation used for and why do we use it?

- **@verify** is a Mockito Method used to check whether a particular method on a mock object was called during the test.
- **verify(mockRepo).save(account);** - verifies that the save() method was called on mockRepo
- **verify(mockRepo, never()).save(account);** - verifies that the save() method was not called.

Task 27:

List down few more annotations that we use for mocking ..

- **@Mock** – Creates a fake/mock object of a class or interface.
- **@InjectMocks** – Creates the object under test and injects mock dependencies into it.
- **@Spy** – Creates a partial mock that uses real methods unless stubbed.
- **@Captor** – Captures method arguments passed to mock objects for verification.
- **@ExtendWith(MockitoExtension.class)** – Enables Mockito annotations in JUnit 5 tests.
- **@MockBean** – Replaces a Spring bean with a mock in the application context.
- **@SpyBean** – Wraps an existing Spring bean with a Mockito spy.